

ATLAS: Adaptive Task-Aware Federated Learning with Heterogeneous LoRA and Split Learning for Edge LLMs

Mahmoud S. Mayaleh*, Zakaria Abou El Houda[†], Lyes Khoukhi*

*Conservatoire national des arts et métiers (CNAM), Paris, France

Email: {mahmoud.mayaleh, lyes.khoukhi}@lecnam.net

[†]INRS, Quebec, Canada

Email: zakaria.abouelhouda@inrs.ca

Abstract—Federated learning (FL) can adapt large language models (LLMs) without centralizing private data, but deployment on heterogeneous edge clients is constrained by task diversity, device variability, and communication cost. Standard task-agnostic aggregation and uniform-rank low-rank adaptation (LoRA) often cause negative transfer and poor resource matching.

This paper presents ATLAS, a task-aware framework for parameter-efficient federated fine-tuning of transformer LLMs on heterogeneous clients. ATLAS combines representation-based task discovery, budget-aware heterogeneous LoRA allocation, state-consistent split federated learning, and LoRA-only graph regularization, together with INT8 LoRA communication and FP32 server-side computation [?], [?], [?], [?], [?], [?]. On a heterogeneous GLUE federation with three tasks, nine clients, and three backbones [?], ATLAS improves the macro-average task score over standard FedAvg by 16.18 points on DistilBERT, 24.83 points on GPT-2, and 6.86 points on Qwen-2.5-0.5B, while reducing LoRA synchronization traffic and eliminating weak-device failures.

Index Terms—Federated learning, large language models, LoRA, split learning, multi-task learning, personalization.

I. INTRODUCTION

FEDERATED learning (FL) enables collaborative model adaptation without centralizing user data, making it attractive for privacy-sensitive on-device personalization. However, federated adaptation of transformer-based large language models (LLMs) on heterogeneous edge devices is constrained by three coupled factors: (i) *task heterogeneity* across clients, (ii) *resource heterogeneity* (memory/compute), and (iii) *communication bottlenecks* that make full-model synchronization impractical.

Task heterogeneity leads to *negative transfer* when a single global model is optimized across diverse NLP tasks, such as sentiment analysis, paraphrase detection, and grammatical acceptability. Classical FL methods such as FedAvg focus on a single global objective, which can degrade both global quality and per-client personalization when client objectives differ substantially [?]. Federated multi-task learning (FMTL) frameworks instead enforce structured sharing across related clients or tasks, often using graph-based couplings to preserve task-specific specialization while still leveraging common structure [?].

In practice, task discovery itself becomes non-trivial when multiple tasks share the same label cardinality. In such settings, gradient-direction fingerprints can collapse toward a common classifier geometry, even though the underlying input distributions remain distinct. This motivates ATLAS to use representation-space fingerprints derived from a frozen backbone, following the broader intuition behind clustered FL and sentence-level semantic embeddings [?], [?], [?].

At the same time, edge devices exhibit strong variation in memory, compute, and network bandwidth, which makes full-model fine-tuning of LLMs impractical in many realistic deployments. Parameter-efficient fine-tuning (PEFT) methods such as LoRA reduce the number of trainable parameters by injecting low-rank adapters into selected layers, thereby lowering memory and often communication costs when only adapters are exchanged [?]. Complementary strategies such as split learning partition the model between clients and a server, offloading deep layers and most of the backpropagation to the server to further reduce client-side load.

However, combining heterogeneous LoRA with split execution introduces two additional systems challenges that are usually hidden in homogeneous FL setups: (i) adapter aggregation must remain well-defined when clients instantiate different ranks, and (ii) sequential access to a shared split server can induce state mismatch between a client’s bottom adapter state and the server-side suffix it interacts with. ATLAS addresses both issues through rank-aware aggregation rules and state-consistent split execution inspired by recent split-FL and personalized FL formulations [?], [?], [?], [?], [?].

Existing work has explored FMTL (e.g., MIRA [?]), heterogeneous PEFT under device constraints (e.g., FAH-QLoRA [?]), and split federated learning (e.g., Split-LoRA/HSplitLoRA/TITANIC [?], [?], [?]), largely in isolation. These approaches do not jointly address representation-based task discovery, heterogeneous-rank aggregation, state-consistent split execution, and graph-based personalization under communication-efficient adapter exchange.

This paper presents **ATLAS**, an adaptive task-aware FL framework for parameter-efficient fine-tuning of transformer-based LLMs on heterogeneous clients. ATLAS integrates four

phases: (i) representation-based task clustering via compact backbone fingerprints, (ii) device- and data-aware heterogeneous LoRA rank allocation under explicit memory budgets, (iii) split federated learning with adaptive split selection, client-specific server-state synchronization, and dual-path aggregation, and (iv) MIRA-style Laplacian regularization over an RBF-kernel task graph derived from those fingerprints.

We implement ATLAS in an end-to-end pipeline and evaluate it on a heterogeneous GLUE federation with three tasks, nine clients, and three backbones. The experimental results show consistent gains over standard FedAvg across all evaluated models, together with lower adapter communication cost and improved robustness for weak devices.

Contributions. ATLAS makes the following contributions:

- Representation-based task discovery: ATLAS derives frozen-backbone fingerprints from averaged [CLS]/pooled hidden states and selects clusters with a multi-metric criterion that penalizes fragmentation, improving task separation when gradients are weakly informative or tasks share the same label structure [?], [?], [?]. A compact PCA target is retained only as an optional compression fallback for high-dimensional fingerprint variants.
- Budget-aware heterogeneous LoRA: ATLAS allocates per-layer LoRA ranks from discrete candidates under explicit adapter-memory budgets, augments attention and FFN projections, and caps ranks for low-resource tasks to avoid over-parameterization on small local datasets [?], [?].
- Dual-path split federated optimization: ATLAS combines heuristic split selection with state-consistent split execution, LoRA-only pre-aggregation personalization, and weighted cluster-level averaging of shared non-LoRA parameters, enabling heterogeneous-rank collaboration without cross-task contamination [?], [?], [?], [?].
- Communication-aware personalization: ATLAS applies a MIRA-style Laplacian over a block-diagonal task graph and couples it with per-channel INT8 LoRA transmission plus FP32 dequantized computation, reducing adapter traffic while preserving the small inter-client differences required by graph regularization [?], [?], [?], [?].

II. RELATED WORK

A. Federated Multi-Task Learning and Personalization

Classical FL algorithms such as FedAvg optimize a single global objective by averaging client updates across all participants [?]. When clients solve different but related NLP tasks, naïve global averaging can induce negative transfer, where updates from one task harm performance on another, ultimately degrading both global quality and per-client personalization.

Federated multi-task learning (FMTL) aims to mitigate this by introducing structured sharing across tasks rather than uniform mixing. MIRA, for instance, introduces graph-based coupling via Laplacian regularization, encouraging similar tasks to remain close in parameter space while preserving per-task specialization [?]. Clustered FL methods such as CFL

and IFCA likewise show that constraining collaboration to task-homogeneous groups is critical when client objectives diverge [?], [?]. However, these methods typically assume either full-model training or clustering signals that remain reliable under homogeneous label semantics.

ATLAS borrows the Laplacian idea but adapts it to (i) LoRA-only personalization, (ii) representation-derived graphs rather than purely gradient-derived ones, and (iii) split execution on constrained devices. The representation view is motivated by sentence-embedding work showing that frozen backbone features often separate tasks before fine-tuning [?].

B. Parameter-Efficient Fine-Tuning under Device Constraints

Parameter-efficient fine-tuning (PEFT) techniques such as LoRA reduce fine-tuning cost by injecting low-rank adapters into selected layers while keeping the base model frozen [?]. This design significantly lowers the number of trainable parameters and can reduce communication when only adapters are shared, but most existing deployments use *uniform* ranks across layers and devices, which can exceed the capacity of low-end hardware and underutilize high-end devices.

Recent work on federated adaptation of LLMs combines LoRA with quantization and heterogeneous configurations to better match device capabilities. FAH-QLoRA, for example, explores joint design of quantization schemes and LoRA adapters, enabling adaptive configurations across clients with different budgets [?]. Quantization-aware PEFT studies such as LLM.int8(), QLoRA, ZeroQuant, and SmoothQuant further show that low-bit communication or storage is viable provided arithmetic is carried out in sufficiently precise formats [?], [?], [?], [?]. Nonetheless, these approaches typically remain task-agnostic at the federation level: aggregation is still performed globally across all clients, and there is no explicit mechanism to prevent cross-task mixing via clustering or graph-based personalization.

ATLAS is task-aware via representation clustering and graph regularization, atop heterogeneous LoRA and communication-aware adapter exchange.

C. Split Learning and Split-PEFT for LLMs

Split learning partitions a model into a client-side prefix and a server-side suffix, allowing clients to compute early-layer activations while offloading deeper layers, loss computation, and most of backpropagation to a powerful server. This paradigm is appealing for edge LLMs because it reduces client memory and compute requirements without fully sacrificing privacy.

Basic split-PEFT methods combine model splitting with LoRA-style adapters to further decrease client-side state. SplitLoRA and HSplitLoRA, for instance, integrate split learning with LoRA adapters to reduce client memory by 50–70% and to enable heterogeneous rank allocation while keeping accuracy close to centralized fine-tuning [?], [?]. SplitFed further highlights the need to keep client and server states synchronized when server-side layers are shared across clients [?].

However, these methods focus primarily on improving feasibility and efficiency of split execution and adapter placement, without modeling task structure or incorporating multi-task personalization objectives.

ATLAS adds task-aware clustering, graph personalization, and dual-path aggregation on top of split-PEFT.

D. System-Level FL Frameworks for LLMs

Several recent frameworks study system-level aspects of FL and split learning for LLMs. VFLAIR-LLM provides an extensible split-LLM benchmark spanning multiple models, datasets, attacks, and defenses, and shows that mid-layer splits often preserve utility while enabling privacy-preserving inference and fine-tuning [?]. TITANIC investigates production-grade FL with LLMs, highlighting challenges such as heterogeneous systems, scaling, and communication optimization in realistic deployments [?].

These frameworks offer valuable insights into practical design trade-offs and provide tooling for evaluation, but they remain largely task-agnostic: clients are typically treated as a homogeneous population, and neither automatic task discovery nor graph-based personalization are tightly integrated into the optimization pipeline. In contrast, ATLAS targets a unified design that couples task-aware clustering, device-aware heterogeneous LoRA allocation, state-consistent split execution, dual-path aggregation, and Laplacian regularization over a fingerprint-derived task graph into a single, end-to-end FL framework for edge LLMs.

III. PROBLEM FORMULATION

We consider a set of clients $\mathcal{K} = \{1, \dots, K\}$ and tasks $\mathcal{T}$. Each client k is associated with a task $t_k \in \mathcal{T}$ and local dataset $D_k = \{(x_i^{(k)}, y_i^{(k)})\}$. Clients are heterogeneous in memory and communication budgets.

A. Split Model and LoRA Adaptation

We start from a pre-trained transformer model split at layer index s into a client-side prefix $f_{\leq s}$ (resides on client), and server-side suffix $f_{>s}$ (on server). For an input x :

$$h = f_{\leq s}(x), \quad u = f_{>s}(h), \quad \hat{y} = h_{t_k}(u), \quad (1)$$

where h_{t_k} is a task-specific classification head. To enable parameter-efficient personalization, each client k trains LoRA adapters A_k with layer-wise ranks $\{r_{\ell,k}\}_{\ell=1}^L$. We distinguish between client-personalized adapter parameters A_k and cluster-shared non-LoRA parameters U_{c_k} (server-side suffix/head parameters synchronized within cluster c_k).

B. FMTL Objective with Graph Regularization

We follow a federated multi-task learning (FMTL) objective:

$$\min_{A,U} J(A,U) = \sum_{k=1}^K f_k(A_k, U_{c_k}) + \eta R(A), \quad (2)$$

where A_k denotes the personalized LoRA parameters for client k , U_{c_k} denotes the shared non-LoRA parameters for cluster

c_k , $\eta \geq 0$ controls coupling strength, and $R(A)$ is a Laplacian regularizer over a task graph:

$$R(A) = \frac{1}{2} \sum_{k=1}^K \sum_{\ell=1}^L a_{k\ell} \|A_k - A_\ell\|_2^2, \quad (3)$$

with adjacency weights $a_{k\ell} \geq 0$ derived from task similarity. Restricting the regularizer to adapters avoids coupling time-inconsistent server-side classifier states in sequential split training [?], [?], [?].

C. Device Constraints

Each client k has a memory budget C_k^{mem} ; the LoRA configuration must satisfy:

$$\sum_{\ell=1}^L 2 d_\ell r_{\ell,k} b_\ell \leq C_k^{\text{mem}}, \quad (4)$$

where d_ℓ is the layer hidden dimension and b_ℓ captures datatype/overhead. In addition, ranks are selected from a discrete candidate set $\mathcal{R} = \{4, 8, 16, 32, 64\}$ and may be upper-bounded by a task-dependent cap $r^{\text{max}}(t_k)$ for low-resource tasks.

IV. ATLAS METHODOLOGY

ATLAS implements a four-phase pipeline: (i) representation fingerprinting and task clustering, (ii) device-aware heterogeneous LoRA rank allocation under explicit budgets, (iii) split federated learning with task-aware aggregation and a heuristic weighted-score split selector, and (iv) Laplacian regularization over a fingerprint-derived task graph. Figure ?? summarizes how these four phases interact within the end-to-end training system.

A. Phase 1: Representation Fingerprinting and Task Clustering

Fingerprint extraction. Fingerprints should separate clients by task semantics before any federated averaging occurs. In ATLAS, each client k therefore computes a *representation fingerprint* from a small number of local examples using the frozen backbone. For encoder models we use the [CLS] token embedding; for decoder-only models we use the analogous pooled final hidden-state representation. These raw sequence embeddings are averaged locally, then ℓ_2 -normalized and mean-centered across clients before cosine-based clustering [?]. We keep PCA compression as an optional fallback for high-dimensional fingerprint variants.

Multi-metric cluster selection. Given fingerprints $\{f_k\}$, ATLAS runs k -means over a constrained candidate range $[k_{\min}, k_{\max}]$ and selects k^* via

$$\text{score}(k) = \alpha_{\text{Sil}} + \beta_{\text{DB}} + \gamma_{\text{CH}} - \lambda \cdot (\#\text{singletons}),$$

where Sil/Davies-Bouldin (DB)/Calinski-Harabasz (CH) are normalized internal metrics ($\alpha, \beta, \gamma = 1$ default; sum normalized), penalizing fragmented clusters. Outputs cluster assignments c_k , used in Phases 3–4.

Algorithm ?? summarizes this three-step pipeline (representation extraction, normalization with optional compression, and multi-metric cluster selection).

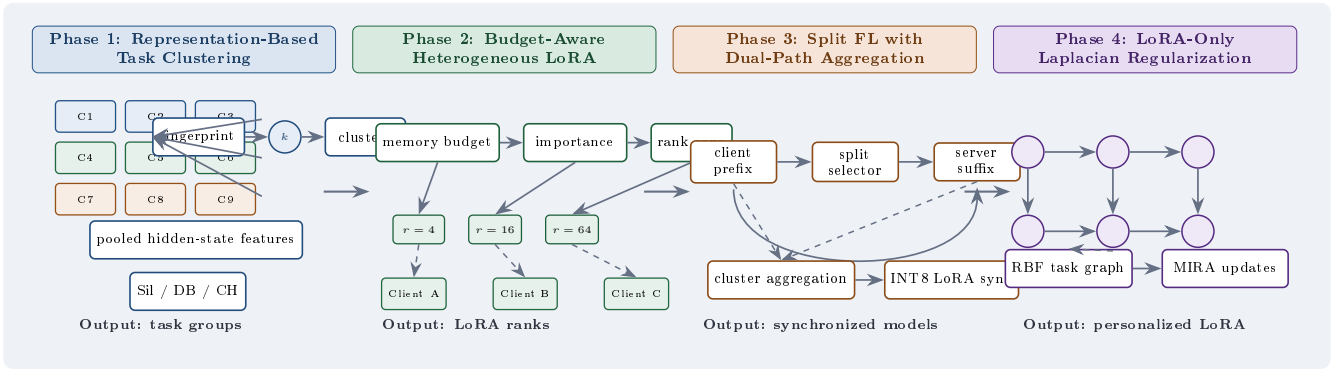


Fig. 1. ATLAS system design. The framework proceeds through four phases: task clustering, device-aware LoRA rank allocation, split federated optimization with cluster aggregation, and LoRA-only graph regularization.

TABLE I
MAIN SYMBOLS USED IN ATLAS.

Symbol	Description
K	Number of clients
$\mathcal{D}_k$	Local dataset of client k
$\mathbf{W}_k^{(t)}$	Trainable parameters of client k at round t
$\mathbf{A}_k^\ell, \mathbf{B}_k^\ell$	LoRA matrices for layer ℓ of client k
r_k^ℓ	LoRA rank for layer ℓ of client k
$\mathbf{f}_k \in \mathbb{R}^d$	Representation fingerprint used for clustering/graph construction
$\mathcal{C}_j$	Task cluster j
c_k	Cluster assignment of client k
$\mathcal{N}_k$	Neighbor set of client k in task graph
$a_{k\ell}$	Adjacency weight between clients k and ℓ
η	Laplacian regularization strength
α	RBF kernel bandwidth
ω_ℓ	Importance weight of layer ℓ
$w_{\min}$	Minimum aggregation weight floor
S_f	Fingerprint samples per client

B. Phase 2: Device-Aware Heterogeneous LoRA Rank Allocation

Each client k is associated with a device memory budget M_k . ATLAS converts it into an *adapter budget* by reserving fractions for the frozen base model, activations, and optimizer state:

$$C_k^{\text{adpt}} = M_k (1 - \alpha_{\text{base}} - \alpha_{\text{act}} - \alpha_{\text{opt}}). \quad (5)$$

LoRA adapter memory for a layer with hidden size d_ℓ and rank $r_{\ell,k}$ is approximated by

$$M_\ell(r_{\ell,k}) \approx 2 d_\ell r_{\ell,k} b, \quad (6)$$

where b depends on numeric precision and implementation overhead.

ATLAS injects LoRA into both attention and FFN projections so that client-side adaptation covers the full transformer block rather than attention alone [?]. It derives per-layer importance weights from local adaptation statistics and allocates ranks from a discrete candidate set (e.g., $\{4, 8, 16, 32, 64\}$) under C_k^{adpt} using an importance-aware, budget-constrained procedure: it finds the largest feasible uniform rank, converts it into a total rank budget, distributes it proportionally to importance, rounds to valid candidates, and downgrades

least-important layers if needed to satisfy the budget. For small-data tasks, a task-adaptive cap $r_k^{\max}(t_k)$ prevents over-parameterized adapters from dominating scarce local supervision [?, ?].

Illustration. For 3 layers with C_k^{adpt} allowing rank-sum ≈ 48 and $\omega = [0.3, 0.5, 0.2]$, proportional allocation yields $r = [16, 24, 8]$ (downgrade least-important if over budget).

C. Phase 3: Split Federated Learning with Heuristic Weighted-Score Split Selection

Split execution. ATLAS partitions the backbone at split layer s into a client-side prefix and server-side suffix. In each local step, the client computes intermediate activations and sends them to the server; the server completes the forward pass, computes the loss, backpropagates through the suffix, and returns gradients w.r.t. the split activations. Clients update only their trainable components while keeping the base model frozen to reduce on-device cost. Because cluster members interact sequentially with a shared server-side suffix, ATLAS restores each client's stored server state at the beginning of its local turn and synchronizes the updated suffix back to that client at the end, following the state-consistency principle of SplitFed-style split FL [?]. We further initialize cluster heads with deterministic Xavier weights and log-prior biases so that the split server begins from a stable, label-aware operating point rather than a degenerate zero-gradient regime [?].

Heuristic split selection (proxy-driven, constrained search). ATLAS evaluates candidate splits within a constrained set (avoiding extreme early/late cuts) and selects the split minimizing a weighted score:

$$\text{score}(s) = \alpha \text{Mem}(s) + \beta \text{Comm}(s) + \gamma \text{Imp}(s) + \delta \text{Bal}(s), \quad (7)$$

where $\alpha, \beta, \gamma, \delta \geq 0$ and $\alpha + \beta + \gamma + \delta = 1$.

Here, α controls the emphasis on respecting client memory budgets, β penalizes communication-heavy splits, γ weights the (optional) importance proxy derived from representation fingerprints, and δ encourages balanced workload between client prefix and server suffix. The weights form a convex

Algorithm 1 Representation-Based Task Clustering in ATLAS

Require: Client set $\mathcal{K} = \{1, \dots, K\}$, frozen backbone $\mathcal{M}_0$, local datasets $\{\mathcal{D}_k\}_{k=1}^K$

Require: Optional projection dimension d , cluster range $[k_{\min}, k_{\max}]$, fingerprint samples per client S_f

Ensure: Task clusters $\mathcal{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_{|\mathcal{C}|}\}$, assignments $\{c_k\}_{k=1}^K$

```

1: Phase 1.1: Representation Fingerprints
2: for each client  $k \in \mathcal{K}$  do
3:   Sample  $S_f$  examples from  $\mathcal{D}_k$ 
4:   for each sampled example or mini-batch  $b$  do
5:     Compute hidden representation  $\mathbf{h}_{k,b} \leftarrow \text{POOL}(\mathcal{M}_0, b)$ 
6:   end for
7:   Average raw embedding:  $\mathbf{e}_k \leftarrow \frac{1}{S_f} \sum_b \mathbf{h}_{k,b}$ 
8: end for
9: Phase 1.2: Normalization / Optional Compression
10: Stack embeddings  $\mathbf{E} = [\mathbf{e}_1, \dots, \mathbf{e}_K]^\top$ 
11: Optionally fit PCA on  $\mathbf{E}$  and obtain projection matrix  $\mathbf{U}_d$  when compression is enabled
12: for each client  $k \in \mathcal{K}$  do
13:   If compression is enabled, set  $\mathbf{f}_k \leftarrow \mathbf{U}_d^\top \mathbf{e}_k$ ; otherwise set  $\mathbf{f}_k \leftarrow \mathbf{e}_k$ 
14:   Normalize fingerprint:  $\mathbf{f}_k \leftarrow \mathbf{f}_k / \|\mathbf{f}_k\|_2$ 
15: end for
16: Phase 1.3: Cluster Selection
17:  $\mathcal{F} \leftarrow [\mathbf{f}_1, \dots, \mathbf{f}_K]^\top$ ,  $\text{best\_score} \leftarrow -\infty$ 
18: for  $n_c = k_{\min}$  to  $k_{\max}$  do
19:   Run  $k$ -means on  $\mathcal{F}$  with  $n_c$  clusters to obtain  $\{\mathcal{C}_j^{(n_c)}\}_{j=1}^{n_c}$ 
20:   Compute internal clustering metrics and singleton count
21:    $s \leftarrow \text{MULTIMETRICSCORE}(\{\mathcal{C}_j^{(n_c)}\}_{j=1}^{n_c})$ 
22:   if  $s > \text{best\_score}$  then
23:      $\text{best\_score} \leftarrow s$ ,  $\mathcal{C} \leftarrow \{\mathcal{C}_j^{(n_c)}\}_{j=1}^{n_c}$ 
24:   end if
25: end for
26: Derive assignments  $c_k = j$  if  $k \in \mathcal{C}_j$ 
27: Return  $\mathcal{C}$  and  $\{c_k\}_{k=1}^K$ 

```

combination ($\alpha + \beta + \gamma + \delta = 1$) and are treated as tunable hyperparameters.

Unless otherwise stated, we set $\alpha = 0.35$, $\beta = 0.30$, $\gamma = 0.25$, and $\delta = 0.10$. These values were selected via preliminary sweeps on a validation configuration to prioritize memory feasibility (α) and communication reduction (β), while still allowing importance (γ) and workload balance (δ) to influence the split choice.

We observed that increasing α beyond 0.4 tended to push splits too early in the network (hurting accuracy), while decreasing β below 0.2 significantly increased activation traffic without accuracy gains.

Here, $\text{Mem}(s)$ and $\text{Comm}(s)$ are proxy estimates (budget check with safety margin and activation-transfer proxy),

$\text{Imp}(s)$ is optional (using representation or gradient signals when available, otherwise generic), and $\text{Bal}(s)$ encourages balanced client/server workload.

Task-aware aggregation and ablations. ATLAS aggregates updates *within each Phase-1 cluster only*, but it does not use a single averaging rule for all trainable parameters. For heterogeneous-rank LoRA adapters, direct FedAvg can suppress higher-rank slices that low-rank clients never trained. ATLAS therefore uses rank-partitioned aggregation whenever adapters must be averaged explicitly (e.g., in non-Laplacian ablations):

$$\bar{A}_g^{(q)} \leftarrow \frac{1}{|\mathcal{S}_{g,q}|} \sum_{k \in \mathcal{S}_{g,q}} A_k^{(q)}, \quad \mathcal{S}_{g,q} = \{k \in g : r_k > q\}, \quad (8)$$

where $A_k^{(q)}$ is the q -th rank slice of client k 's adapter and only clients that instantiate that slice contribute to its average. In the full atlas path, adapters are instead personalized first by the Laplacian update on raw local weights, while shared non-LoRA parameters are averaged separately:

$$\bar{U}_g \leftarrow \sum_{k \in g} \tilde{\pi}_k U_k, \quad \tilde{\pi}_k = \frac{\max(\pi_k, w_{\min})}{\sum_{j \in g} \max(\pi_j, w_{\min})}, \quad (9)$$

with π_k denoting data- or performance-based client weights and $w_{\min}$ a minimum floor that prevents slow-converging clients from being eliminated by the aggregation rule [?], [?].

Communication-aware adapter exchange. Split execution introduces activation transfer, but the primary communication metric reported in the main ATLAS comparisons is *LoRA synchronization traffic*, since this is the method-dependent overhead directly controlled by heterogeneous rank allocation and INT8 adapter exchange. LoRA parameters are quantized per output channel to INT8 for transmission, but they are always dequantized back to FP32 before any averaging or Laplacian arithmetic. This preserves the small inter-client differences required by graph regularization while retaining nearly $4\times$ compression relative to FP32 adapter exchange [?], [?], [?].

This yields clean, implementation-faithful ablations: (i) `fedavg_cluster`: cluster-restricted averaging only (no split personalization, no Laplacian), (ii) `atlas_no_laplacian`: split FL with heterogeneous ranks and rank-aware aggregation, without graph regularization, (iii) `atlas`: dual-path aggregation with LoRA-only Laplacian personalization and shared-parameter averaging.

D. Phase 4: Laplacian Regularization over a Fingerprint-Derived Task Graph

ATLAS constructs a task graph from fingerprints and applies a MIRA-style Laplacian update for personalization. For clients k and ℓ , adjacency weights are computed using an RBF kernel:

$$a_{k\ell} = \exp(-\alpha \|f_k - f_\ell\|_2^2), \quad (10)$$

then row-normalized so that $\sum_\ell a_{k\ell} = 1$.

By default, ATLAS enforces a block-diagonal graph using the selected clusters, thereby removing cross-cluster edges. In

Algorithm 2 Task Graph Construction with RBF Kernel

Require: Fingerprints $\{\mathbf{f}_k\}_{k=1}^K$, cluster assignments $\{c_k\}_{k=1}^K$
Require: RBF bandwidth α , neighbor budget K_{nn}
Ensure: Adjacency matrix $\mathbf{A} = [a_{k\ell}]_{K \times K}$

- 1: Initialize $\mathbf{A} \leftarrow \mathbf{0}_{K \times K}$
- 2: **for** each client $k \in \mathcal{K}$ **do**
- 3: $\mathcal{T}_k \leftarrow \{\ell : c_\ell = c_k\}$ ▷ Same-cluster clients
- 4: Compute similarities $s_{k\ell} = \exp(-\alpha \|\mathbf{f}_k - \mathbf{f}_\ell\|_2^2)$ for $\ell \in \mathcal{T}_k \setminus \{k\}$
- 5: $\mathcal{N}_k \leftarrow \text{TopK}(\{s_{k\ell}\}, K_{\text{nn}})$
- 6: Normalize: $a_{k\ell} \leftarrow s_{k\ell} / \sum_{j \in \mathcal{N}_k} s_{kj}$ for $\ell \in \mathcal{N}_k$
- 7: **end for**
- 8: Symmetrize: $a_{k\ell} \leftarrow (a_{k\ell} + a_{\ell k})/2$ for all k, ℓ
- 9: **Return** $\mathbf{A}$

benchmark settings where task identifiers are available, the graph can be further filtered to same-task neighbors as a safeguard against impure cross-task couplings [?], [?]. Singletons are connected to K_{nn} nearest neighbors for connectivity.

Given client adapter parameters A_k , ATLAS applies the Laplacian to the raw local LoRA weights *before* any cluster averaging:

$$A_k \leftarrow A_k - \eta \sum_{\ell} a_{k\ell} (A_k - A_{\ell}), \quad (11)$$

where η controls the coupling strength. Applying the update on pre-aggregation local adapters avoids the degenerate zero-regularization case that occurs if all cluster members are first averaged to the same point [?], [?], [?]. This nudges similar clients to align while preserving task-specific personalization.

Heterogeneous rank handling. To align adapters across clients with varying ranks $r_{k\ell} \neq r_{k'\ell}$, ATLAS computes the Laplacian in the shared low-rank subspace only. For a client pair (k, ℓ) , let $r_{\min} = \min(r_k, r_\ell)$; the LoRA difference is evaluated on the first $r_{\min}$ rows/columns and the resulting update is zero-padded back to client k 's native rank. This preserves high-rank client capacity while regularizing the common subspace shared with its neighbors.

V. EXPERIMENTS

A. Tasks, Data, and Client Federation

We evaluate ATLAS in a heterogeneous multi-task FL setting using three GLUE sentence classification tasks: SST-2 (sentiment analysis), MRPC (paraphrase detection), and QNLI (question natural language inference) [?]. Each client is assigned to exactly one task and trains only on its local data; no raw data are shared.

We simulate a federation of $K = 9$ clients (3 clients per task) with heterogeneous device profiles emulating 2–16 GB memory budgets. Unless otherwise noted, each client uses up to 3,000 local training samples (subsampled if necessary) to control runtime variance across tasks and models.

Algorithm 3 Laplacian Regularization with Heterogeneous LoRA Ranks

Require: Client adapters $\{\mathbf{A}_k^{(t)}\}_{k=1}^K$ after local training at round t
Require: Task graph $\mathcal{G} = (\mathcal{K}, \mathcal{E})$ with weights $\{a_{k\ell}\}$
Require: Regularization strength η
Require: Layer importance weights $\{\omega_\ell\}_{\ell \in \mathcal{L}}$
Ensure: Updated adapters $\{\mathbf{A}_k^{(t+1)}\}_{k=1}^K$

- 1: Each $\mathbf{A}_k^{(t)} = \{\theta_k^{\ell, (t)}\}_{\ell \in \mathcal{L}}$ ▷ LoRA blocks
- 2: **for** each client $k \in \mathcal{K}$ **do**
- 3: $\mathcal{N}_k \leftarrow \{\ell : (k, \ell) \in \mathcal{E}\}$ ▷ Neighbors
- 4: **if** $|\mathcal{N}_k| = 0$ **then**
- 5: $\mathbf{A}_k^{(t+1)} \leftarrow \mathbf{A}_k^{(t)}$ ▷ No Laplacian update
- 6: **continue**
- 7: **end if**
- 8: **for** each layer $\ell \in \mathcal{L}$ **do**
- 9: $\Delta \theta_k^\ell \leftarrow 0$
- 10: **for** each neighbor $j \in \mathcal{N}_k$ **do**
- 11: $r_{\min} \leftarrow \min(r_{k, \ell}, r_{j, \ell})$
- 12: $(\hat{\theta}_k^\ell, \hat{\theta}_j^\ell) \leftarrow \text{SHARED SUBSPACE}(\theta_k^{\ell, (t)}, \theta_j^{\ell, (t)}, r_{\min})$
- 13: $\lambda_{kj} \leftarrow \omega_\ell \cdot a_{kj}$
- 14: $\Delta \theta_k^\ell \leftarrow \Delta \theta_k^\ell + \lambda_{kj} (\hat{\theta}_k^\ell - \hat{\theta}_j^\ell)$
- 15: **end for**
- 16: $\theta_k^{\ell, (t+1)} \leftarrow \theta_k^{\ell, (t)} - \eta \cdot \Delta \theta_k^\ell$
- 17: **end for**
- 18: $\mathbf{A}_k^{(t+1)} \leftarrow \{\theta_k^{\ell, (t+1)}\}_{\ell \in \mathcal{L}}$
- 19: **end for**
- 20: **Return** $\{\mathbf{A}_k^{(t+1)}\}_{k=1}^K$

TABLE II
FEDERATION AND DATASET CONFIGURATION USED IN THE EXPERIMENTS.

Task	Clients	Metric	Max samples/client
SST-2	3	Accuracy	3000
MRPC	3	F1	3000
QNLI	3	Accuracy	3000

B. Models

We consider three backbones spanning encoder and decoder architectures: *DistilBERT-base-uncased* (distilbert-base-uncased), *GPT-2* (gpt2), and *Qwen2.5-0.5B* (Qwen/Qwen2.5-0.5B).

C. Training Protocol

Unless otherwise stated, we fine-tune LoRA adapters together with the cluster-specific server head while keeping the backbone frozen. Fingerprints are extracted from frozen-backbone sequence embeddings, cluster heads are initialized with deterministic Xavier weights plus log-prior biases, and the Laplacian is enabled after a one-round warmup. We run federated training for 10 communication rounds with 3 local epochs per round. The main comparisons are reported over seeds $\{42, 123, 456\}$.

TABLE III
GLOBAL HYPERPARAMETERS SHARED ACROSS THE COMPLETED
EXPERIMENTS.

Hyperparameter	Value
Global rounds	10
Local epochs/round	2
Max samples/client	3000
Seeds	{42, 123, 456}
LoRA rank candidates	{4, 8, 16, 32, 64}
Split-layer candidate set	architecture-dependent middle-layer window
Laplacian warmup rounds	1

D. Baselines and Progressive Ablation Study

To compare “everything” while keeping the study interpretable, we adopt a progressive ablation that disables ATLAS components one at a time:

- **standard_fl**: standard FedAvg across all clients (no task clustering, no heterogeneous ranks, no split learning, no Laplacian).
- **fedavg_cluster**: task-aware FedAvg by aggregating only within Phase 1 clusters (no heterogeneous ranks, no split learning, no Laplacian).
- **atlas_no_laplacian**: full ATLAS pipeline (task clustering + heterogeneous ranks + split learning) without Laplacian regularization.
- **atlas**: full ATLAS system including Laplacian regularization.

E. Metrics

We report task metrics on each client validation split: accuracy (SST-2), F1 (MRPC), and accuracy (QNLI). To summarize performance across the heterogeneous federation, we use the macro-average of these three task metrics. We additionally report LoRA synchronization traffic and the number of weak-device clients dropped by the training system. In the recorded experiment logs, a dropped weak device corresponds to an entry in the `weak_device_dropped` structure, providing a direct robustness indicator for heterogeneous participation.

F. Main Results

Table ?? summarizes the main quantitative results. ATLAS outperforms standard FedAvg on all three backbones. On DistilBERT, ATLAS improves the macro-average task score from 0.6214 ± 0.0125 to 0.7832 ± 0.0078 , a gain of $+0.1618$. On GPT-2, the gain is larger, from 0.5605 ± 0.0071 to 0.8088 ± 0.0158 , corresponding to $+0.2483$. On Qwen-2.5-0.5B, ATLAS improves the macro-average task score from 0.7882 ± 0.0247 to 0.8568 ± 0.0054 , a gain of $+0.0686$. For GPT-2, all four methods are evaluated over the same three seeds, which makes the ablation ordering directly comparable under matched mean $\pm$ std reporting.

The gains are consistent at the per-task level. For DistilBERT, ATLAS improves SST-2, MRPC, and QNLI by 0.1949, 0.2875, and 0.1030, respectively, relative to standard FedAvg. For GPT-2, the corresponding improvements are 0.2112, 0.2828, and 0.2821. For Qwen-2.5-0.5B, the gains

are 0.0738, 0.0789, and 0.0531. Figure ?? confirms that these improvements are systematic across backbone families, while Figure ?? shows that ATLAS improves each task-specific metric rather than merely redistributing performance across tasks.

G. Communication and Robustness

ATLAS also improves communication efficiency. Relative to standard FedAvg, LoRA synchronization traffic drops from 889.3 MB to 611.4 MB on DistilBERT (31.3% reduction), from 827.4 MB to 554.7 MB on GPT-2 (33.0%), and from 4070.9 MB to 1154.0 MB on Qwen-2.5-0.5B (71.7%). The largest absolute reduction occurs on Qwen, where ATLAS reduces synchronization traffic by 2916.9 MB while still improving the macro-average task score. Figure ?? makes the systems point more clearly: ATLAS shifts the operating point toward the desirable high-score, low-traffic region rather than trading one objective for the other. The GPT-2 ablation follows the same pattern, with each additional ATLAS component moving the method toward a better communication-performance frontier.

Robustness to device heterogeneity also improves materially. Table ?? replaces the per-client failure figure with the more decision-relevant summary for a systems paper: whether the weakest clients remain trainable. In all three backbone families, standard FedAvg drops both weak-device clients, while ATLAS retains both. Figure ?? further indicates that the advantage is not only a late-round effect: ATLAS converges toward stronger final solutions while avoiding the instability associated with weak-client attrition in the baseline. This matters because a heterogeneous FL method that achieves high accuracy only by excluding weaker devices does not satisfy the motivating deployment constraint.

H. Runtime Analysis

The JSON logs also record per-round wall-clock time and total run time, which lets us qualify the systems gains beyond communication volume alone. Table ?? reports the mean runtime over the three seeds for the main ATLAS versus standard-FedAvg comparisons. As expected, ATLAS is slower in raw wall-clock time because it performs additional clustering-aware bookkeeping, split-server coordination, and Laplacian personalization. On DistilBERT, the mean total runtime increases from 1409.0 s to 1699.4 s; on GPT-2, from 3183.8 s to 5491.3 s; and on Qwen-2.5-0.5B, from 11591.2 s to 15055.1 s. The key point is therefore not that ATLAS reduces wall-clock time, but that it improves task score, communication efficiency, and weak-device retention at a measurable computational overhead that is now explicitly documented.

I. Ablation on GPT-2

The GPT-2 ablation provides component-level evidence for the ATLAS design. Table ?? shows a clear progression as ATLAS components are enabled. Relative to standard FedAvg, `fedavg_cluster` improves the macro-average task score

TABLE IV
METHOD COMPARISON MATRIX USED IN THE EXPERIMENTS.

Method	Clustering	Het. ranks	Split	Laplacian	Global mixing
standard_fl	–	–	–	–	✓
fedavg_cluster	✓	–	–	–	–
atlas_no_laplacian	✓	✓	✓	–	–
atlas	✓	✓	✓	✓	–

TABLE V
MAIN RESULTS ACROSS BACKBONES.

Backbone	Method	Macro Avg. Score	SST-2	MRPC (F1)	QNLI	Weak Devs Dropped	LoRA Comm. (MB)
DistilBERT	Standard FedAvg	0.6214 ± 0.0125	0.6705	0.4423	0.6514	2	889.3
	ATLAS	0.7832 ± 0.0078	0.8654	0.7298	0.7544	0	611.4
GPT-2	Standard FedAvg	0.5605 ± 0.0071	0.6858	0.4623	0.5335	2	827.4
	FedAvg-Cluster	0.6962 ± 0.0016	0.6358	0.6046	0.6958	2	685.8
	ATLAS w/o Laplacian	0.7881 ± 0.0002	0.8237	0.6548	0.7456	0	587.5
	ATLAS	0.8088 ± 0.0158	0.8970	0.7451	0.8156	0	554.7
Qwen-2.5-0.5B	Standard FedAvg	0.7882 ± 0.0247	0.8295	0.7234	0.8223	2	4,070.9
	ATLAS	0.8568 ± 0.0054	0.9033	0.8023	0.8754	0	1,154.0

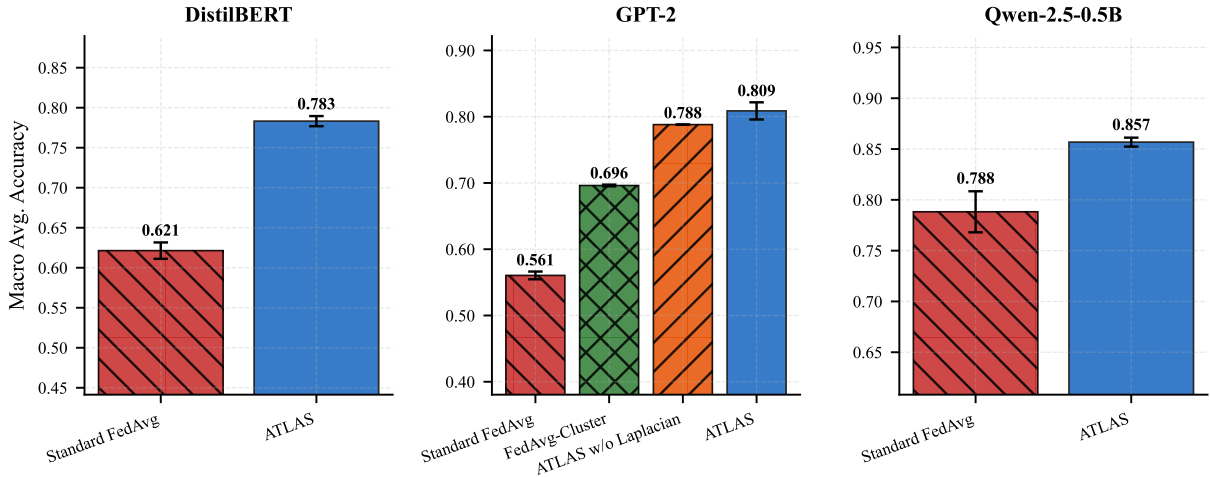


Fig. 2. Macro-average task score by method and backbone. Error bars denote standard deviation across seeds when multiple runs are available.

TABLE VI
EFFICIENCY AND ROBUSTNESS GAINS OF ATLAS OVER STANDARD FEDAVG.

Backbone	Δ Macro Score	Comm. Reduction	Weak Clients Recovered
DistilBERT	+0.1618	31.3%	2
GPT-2	+0.2483	33.0%	2
Qwen-2.5-0.5B	+0.0686	71.7%	2

TABLE VII
WEAK-DEVICE RETENTION UNDER HETEROGENEOUS PARTICIPATION.
EACH BACKBONE INCLUDES TWO WEAK-DEVICE CLIENTS IN THE 9-CLIENT FEDERATION.

Backbone	Std. FedAvg Drops	ATLAS Drops	ATLAS Retention
DistilBERT	2/2	0/2	100%
GPT-2	2/2	0/2	100%
Qwen-2.5-0.5B	2/2	0/2	100%

from 0.5605 ± 0.0071 to 0.6962 ± 0.0016 , indicating that task-consistent mixing alone already mitigates substantial negative transfer. Adding heterogeneous ranks and split learning through `atlas_no_laplacian` raises the macro-average task score further to 0.7881 ± 0.0002 , showing that most of the gain is preserved even before graph regularization is applied. The full ATLAS run reaches 0.8088 ± 0.0158 , yielding an addi-

tional $+0.0207$ improvement over `atlas_no_laplacian`. Figure ?? visualizes the same ordering and shows that the Laplacian gain is accompanied by simultaneous improvements in all reported GPT-2 metrics, not only in the macro-average summary.

The ordering remains coherent throughout the GPT-2 ab-

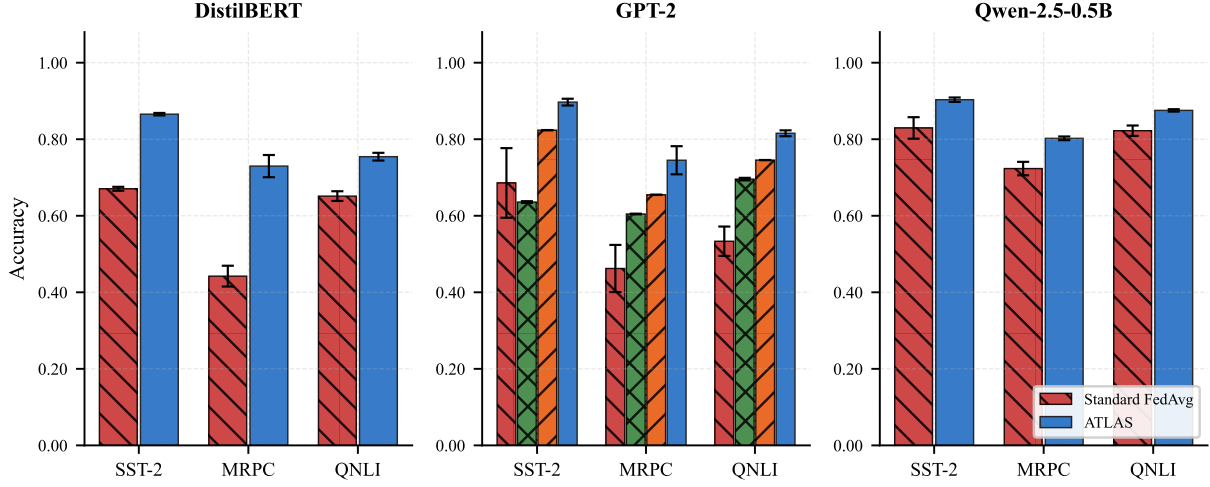


Fig. 3. Per-task performance across backbones. ATLAS improves SST-2 accuracy, MRPC F1, and QNLI accuracy consistently; on GPT-2, the full method also outperforms both `fedavg_cluster` and `atlas_no_laplacian`.

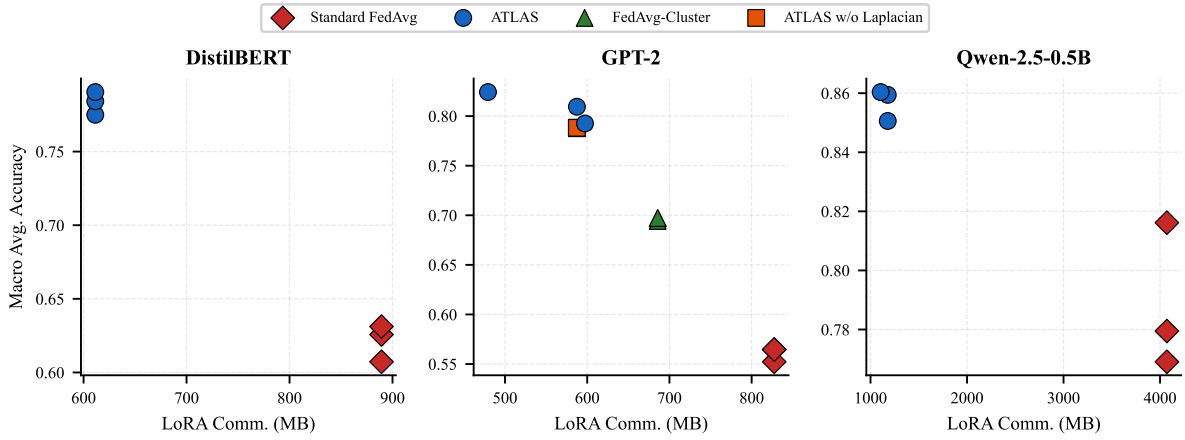


Fig. 4. Communication-performance trade-off. Each point reports macro-average task score versus LoRA synchronization traffic. Across backbones, ATLAS moves the operating point toward higher task score at lower communication cost, and the GPT-2 ablation follows the same trend as ATLAS components are enabled.

TABLE VIII
RUNTIME SUMMARY FROM JSON WALL-CLOCK LOGS (MEAN $\pm$ STD OVER THREE SEEDS).

Backbone	Method	Total Time (s)
DistilBERT	Standard FedAvg	1409.0 $\pm$ 58.3
DistilBERT	ATLAS	1699.4 $\pm$ 14.1
GPT-2	Standard FedAvg	3183.8 $\pm$ 127.9
GPT-2	ATLAS	5491.3 $\pm$ 1132.3
Qwen-2.5-0.5B	Standard FedAvg	11591.2 $\pm$ 4.1
Qwen-2.5-0.5B	ATLAS	15055.1 $\pm$ 384.4

TABLE IX
GPT-2 ABLATION STUDY. THE FULL ATLAS PIPELINE YIELDS THE STRONGEST OVERALL RESULT, AND THE LAPLACIAN CONTRIBUTES AN ADDITIONAL +0.0207 MACRO-AVERAGE TASK SCORE OVER ATLAS WITHOUT LAPLACIAN.

Method	Macro Avg.	SST-2	MRPC (F1)	QNLI
Standard FedAvg	0.5605 $\pm$ 0.0071	0.6858	0.4623	0.5335
FedAvg-Cluster	0.6962 $\pm$ 0.0016	0.6358	0.6046	0.6958
ATLAS w/o Lap	0.7881 $\pm$ 0.0002	0.8237	0.6548	0.7456
ATLAS	0.8088 $\pm$ 0.0158	0.8970	0.7451	0.8156

J. Device-Aware Adapter Allocation

lation: clustering helps, the full heterogeneous split pipeline helps more, and the Laplacian provides an additional incremental gain on top of an already strong non-Laplacian system.

The systems argument of ATLAS depends not only on accuracy but also on whether the method actually adapts model capacity to device constraints. Figure ?? shows that this behavior is present in the GPT-2 runs. Standard FedAvg

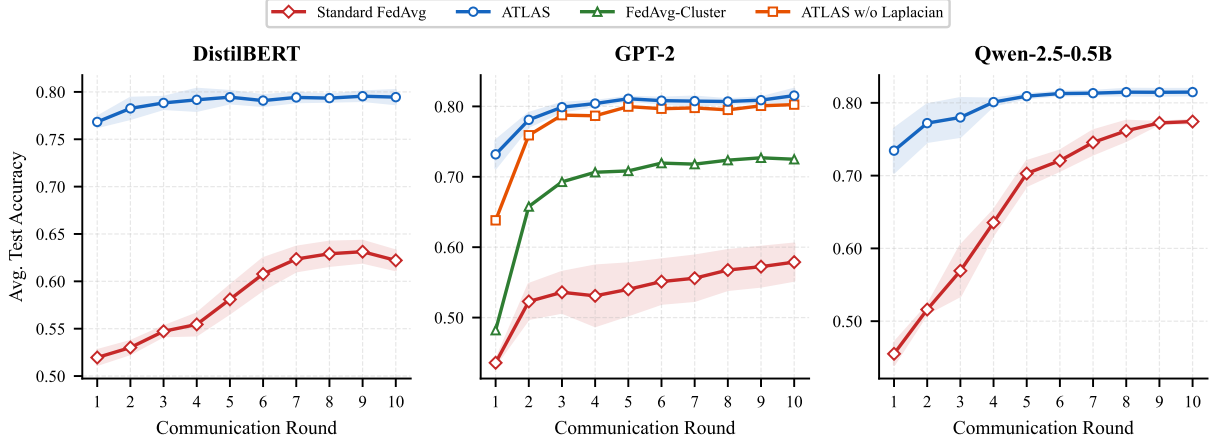


Fig. 5. Convergence curves for the evaluated backbones. ATLAS converges to a stronger final solution across all three models.

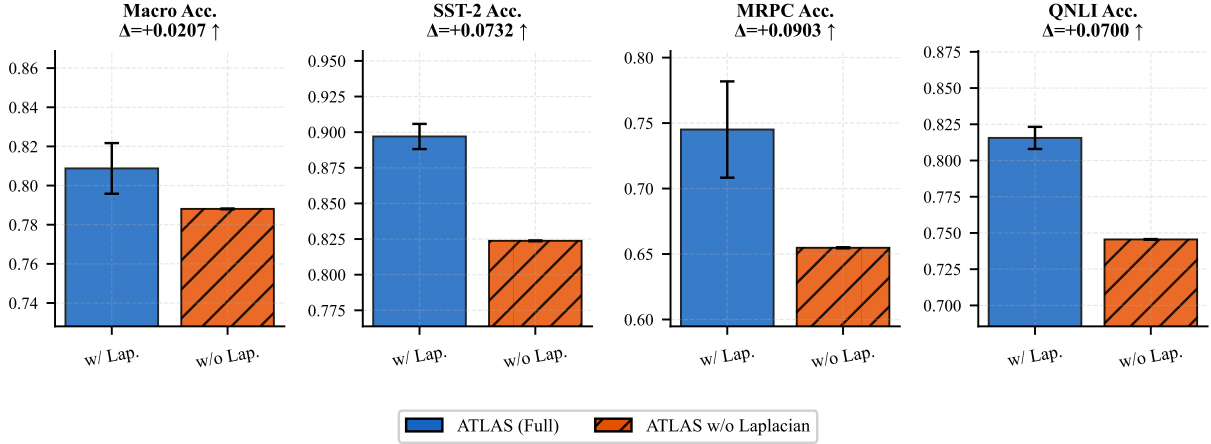


Fig. 6. GPT-2 ablation. Restricting aggregation to task-consistent clusters yields a large improvement over standard FedAvg; adding heterogeneous split training yields a second large improvement; the Laplacian then provides an additional, consistent refinement across the reported metrics.

keeps the adapter budget fixed at roughly 221K trainable LoRA parameters for all device tiers. In contrast, ATLAS assigns substantially smaller adapters to weak devices and larger adapters to strong devices, ranging from about 94K parameters on 2 GB devices to about 770K on 16 GB devices. This monotonic capacity scaling is consistent with the design goal of preserving feasibility on weak clients without wasting available memory on strong ones.

VI. DISCUSSION

The results indicate that ATLAS improves the quality-communication-robustness frontier rather than a single metric in isolation. Across backbones, it increases macro-average task score, reduces LoRA synchronization traffic, and retains weak devices that standard FedAvg repeatedly drops. The GPT-2 ablation shows that this gain is cumulative: task-aware clustering removes much of the negative transfer, heterogeneous split adaptation adds a second improvement, and Laplacian personalization provides a further refinement.

The main trade-off is runtime. ATLAS is slower in wall-clock time than standard FedAvg because clustering-aware coordination, split-state synchronization, and graph-based personalization add overhead. The systems claim is therefore not lower runtime, but a better operating point: higher task score, lower adapter communication, and stronger participation robustness at a measurable computational cost.

The weak-device results are especially important for practical edge deployment. A heterogeneous FL method should not depend on excluding the least capable clients to remain competitive. In the reported federation, standard FedAvg repeatedly drops both weak devices, whereas ATLAS retains them while still improving the aggregate score. This pattern helps explain why the components work well together: clustering limits harmful cross-task mixing, heterogeneous LoRA ranks avoid uniform over-provisioning, split execution reduces client burden, and the Laplacian personalizes within the discovered task structure instead of smoothing across unrelated clients.

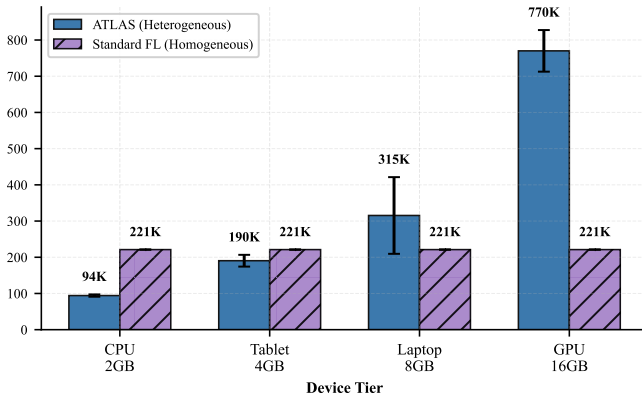


Fig. 7. Device-aware adapter allocation for GPT-2. ATLAS scales trainable LoRA capacity with device tier, whereas standard FedAvg enforces the same adapter size on all clients. Error bars denote standard deviation across the three seeds.

TABLE X
QUALITATIVE COMPARISON WITH CLOSELY RELATED FL/LLM SYSTEMS.

Work	Task-aware	Het. PEFT	Split	Graph reg.
MIRA [?]	✓	–	–	✓
FAH-QLoRA [?]	–	✓	–	–
SplitLoRA [?]	–	–	✓	–
HSplitLoRA [?]	–	✓	✓	–
TITANIC [?]	–	✓	–	–
ATLAS (this work)	✓	✓	✓	✓

A. Comparison to Prior Work

Table ?? positions ATLAS against the most relevant prior systems. We deliberately keep this comparison qualitative because the completed experiments in this paper target a multi-task heterogeneous federation, whereas prior systems often emphasize only one axis such as graph regularization, heterogeneous PEFT, or split execution. A faithful quantitative head-to-head comparison would require reproducing those systems under the same federation and backbone settings.

Relative to MIRA, the main distinction is that ATLAS applies graph regularization only to personalized LoRA adapters and integrates it with split execution and device-aware heterogeneous ranks. Relative to FAH-QLoRA, the contribution is not only heterogeneous PEFT but also task-aware collaboration and graph-based personalization. Relative to SplitLoRA and HSplitLoRA, the contribution is not only split feasibility, but the explicit treatment of task structure, rank-aware aggregation, and task-constrained personalization. This combination is what allows ATLAS to address the coupled statistical and systems heterogeneity present in realistic edge federations.

The novelty of ATLAS therefore lies less in introducing a single entirely new primitive than in the system-level integration of several complementary ideas into one coherent pipeline. Representation-based task discovery, heterogeneous LoRA allocation, split execution, and graph regularization have each appeared in partial form in prior literature, but ATLAS is, to our knowledge, the first framework to combine

them in a unified design targeted at heterogeneous edge LLM adaptation. We view this as a practical rather than purely taxonomic contribution: the value of the framework comes from making these components compatible under one optimization and communication protocol.

B. Limitations and Threats to Validity

The current study has five main limitations. First, the evaluation is conducted on a moderate federation with nine clients and three GLUE classification tasks, so the present results should be read as a proof-of-concept rather than a full scalability study. In particular, while Phase 1 clustering and Phase 4 graph construction are lightweight at this scale, much larger federations with hundreds or thousands of clients would likely require approximate nearest-neighbor graph construction, sparse connectivity constraints, or hierarchical clustering to keep similarity estimation and synchronization overhead manageable.

Second, the framework has several interacting design choices, including the split-selection score weights, Laplacian strength η , RBF bandwidth α , rank caps, and cut-layer placement. The current paper fixes these values to defaults that worked consistently across the completed runs, but it does not yet provide a comprehensive sensitivity analysis.

Third, reproducibility is helped by fixed seeds, saved JSON logs, and released scripts, but the full ATLAS pipeline remains more complex than a standard FedAvg baseline because several phases interact sequentially. A stronger reproducibility package should therefore include a reduced benchmark suite for sanity checking each phase in isolation, together with ablations that show which conclusions are stable across alternate clustering and split configurations.

Fourth, privacy is only partial: ATLAS keeps raw data local, but the split-learning path exchanges intermediate activations (“smashed data”), which can still leak information through inversion or property-inference attacks [?], [?]. The current paper does not quantify this privacy-utility trade-off. Promising mitigation directions include differential privacy or calibrated noise on transmitted activations, secure aggregation or encryption for adapter exchange, and empirical evaluation under activation-reconstruction attacks to determine how much privacy is lost at different split depths.

Fifth, while the JSON logs provide wall-clock runtime, the paper still does not report client FLOPs, energy, or bandwidth-latency breakdowns by device tier. These measurements are necessary for a deployment-oriented assessment and remain important directions for future work.

VII. CONCLUSION

This paper presented ATLAS, a unified framework for federated adaptation of transformer-based LLMs over heterogeneous edge clients. ATLAS combines representation-based task discovery, budget-aware heterogeneous LoRA allocation, state-consistent split federated learning, and LoRA-only graph regularization within a single end-to-end pipeline. Across

DistilBERT, GPT-2, and Qwen-2.5-0.5B, the framework consistently improves macro-average task score over standard FedAvg while reducing LoRA synchronization traffic and eliminating weak-device failures in the evaluated federation.

The main conclusion is that statistical heterogeneity and systems heterogeneity should be handled jointly rather than as separate optimization problems. In ATLAS, task-aware clustering limits negative transfer, heterogeneous adapter allocation matches model capacity to device budgets, split execution improves feasibility for constrained clients, and graph-based personalization refines adaptation within task-consistent neighborhoods. The resulting gains are therefore not attributable to a single isolated mechanism, but to the interaction between these components.

At the same time, the current evidence has clear boundaries: the federation scale is moderate, the workloads are still narrow, the split-learning path introduces privacy exposure through exchanged activations, and the hyperparameter study is not yet exhaustive. The most important next step is not adding one more component, but stress-testing the existing design through larger federations, broader generative workloads, explicit sensitivity analyses over η , α , and split-selection weights, and privacy-enhanced split protocols with measured privacy-utility trade-offs. If these follow-up studies confirm the present trends, ATLAS could serve as a useful reference architecture for practical on-device LLM adaptation in heterogeneous federated settings.